JS & CSS — Lessons Learned (Font/Theme/Color Settings Feature)

A running list of real bugs I hit while building this, and the underlying concept behind each one. Read this before starting a new project.

---

1. CSS cascade order beats JS class-add order

Mistake:

```
bodyContent.classList.add("font-alexandria");
// but <body> already had class="font-tajawal" hardcoded
```

Alexandria never showed up — Tajawal kept winning, even though `font-alexandria` was added *after* it in JS.

Concept: When two classes on the same element have equal CSS specificity, the browser applies whichever rule appears **last in the stylesheet** — not whichever class was added last in the DOM/JS. `classList.add()` order is irrelevant to the cascade.

Fix: Before adding the class you want to win, explicitly `remove()` every competing class first. Never assume "added later" means "wins."

---

2. `e.target` is not always the element you

---

clicked

Mistake:

```
fontOptionsButtons.addEventListener("click", function(e){
  if(e.target.getAttribute("data-font") === "alexandria")
```

Clicking the button worked *sometimes* — only if you hit the exact padding area. Clicking the label text or icon inside it did nothing.

Concept: `e.target` is the *exact* element the click landed on, which can be any nested child (`<span>`, `<div>`, icon) — not necessarily the element you attached the listener to or the one with the data you need.

Fix: Use `e.target.closest(".my-button-class")` to walk up to the actual element you care about, regardless of which inner child was clicked. Always follow it with a guard: `if(!target) return;` (closest returns `null` if nothing matched, and calling methods on `null` throws).

---

3. `NodeList.length` vs a single `Element`

Mistake:

```
var fontOptionsButtons = document.getElementById("font-di
for(var i = 0; i < fontOptionsButtons.length; i++){ ... }
```

`i < undefined` is always `false` → the loop body never runs, not even once, and it fails completely silently (no error).

Concept: `getElementById` returns a single `Element`. `querySelectorAll` returns a `NodeList`. Only the `NodeList` has a `.length` to loop over. Mixing the two up gives no error — just a loop that quietly does nothing.

Fix: Know which variable is "the one" vs "the many," and sanity-check

by logging `.length` while debugging if a loop seems to do nothing.

---

4. Comparing a counter to the list itself, not `list.length`

Mistake:

```
for(var i = 0; i < colorOptions; i++){ ... }   // colorOp
```

Same silent failure as #3, but inverted: forgot `.length` entirely. Effect: old selections never got cleared, so highlights piled up on multiple buttons at once instead of moving to the new one.

Concept: A `for` loop's condition must compare against a **number**. Comparing a number to an object/array is (almost) always `false` in JS — it doesn't throw, it just silently skips the loop.

Fix: Whenever a loop "does nothing," the very first thing to check is whether the condition is actually comparing to a `.length`.

---

5. Resetting state *before* checking it makes a branch unreachable

Mistake:

```
if(target.getAttribute("data-font") === "alexandria"){
  deactiveFontButton();  // sets aria-checked="false" on
  if(target.getAttribute("aria-checked") === "false"){ ..
  else if(target.getAttribute("aria-checked") === "true")
```

The "toggle off" branch became dead code, because the reset function ran first and already forced the value the `if` was about to check.

Concept: Order of operations matters when a function you call has a side effect on the very state you're about to branch on. Read the current state into a variable *before* calling anything that might change it, if you still need the "before" value afterward.

Fix:

```
var wasChecked = target.getAttribute("aria-checked") ===
deactiveFontButton();
if(!wasChecked){ ... }
```

---

6. `localStorage.getItem()` returns `null` on first visit — always have a fallback

Mistake:

```
var fontName = localStorage.getItem("font"); // null on f
if(fontName === "alexandria"){ ... }
else if(fontName === "tajawal"){ ... }
else if(fontName === "cairo"){ ... }
// null matches none of these → nothing happens
```

Result: the page rendered with a default font (from the raw HTML), but no button was ever marked as "selected" — a mismatch between what's applied and what the UI shows.

Concept: `localStorage.getItem()` returns `null`, not an empty string or a sensible default, when the key was never set. Every `if/else if` chain reading from storage needs to account for that first-ever-visit case.

Fix: `localStorage.getItem("font") || "tajawal"` — fall back to whatever the real default actually is (match it to what's hardcoded in the HTML/CSS).

---

7. Two controls, two separate state flags = desync bugs

Mistake: The settings sidebar's open/close button tracked its own `aria-expanded`, while the close (X) button tracked a *different* element's `aria-hidden` — two disconnected sources of truth for the same visual state. Result: had to click twice to see any effect, because the two flags could drift out of sync with each other and with the sidebar's actual class.

Concept: When multiple controls affect the same piece of UI state, tracking that state in more than one place (different attributes on different elements) lets them contradict each other.

Fix: Pick **one** source of truth — usually the actual DOM state of the thing being controlled (e.g. `sidebar.classList.contains("translate-x-full")`) — and have every control read/write through a single shared function.

---

8. The live DOM (DevTools → Elements) is not the same as your source file

Observation: DevTools showed buttons inside `#theme-colors-grid` that didn't exist anywhere in the actual `.html`/`.js` files.

Concept: The **Elements** panel shows the current, *live* DOM — after every script has run and possibly mutated it via `innerHTML`, `appendChild`, etc. The source file on disk never changes just because JS edited the page in the browser. "View Source" (Ctrl+U) / the raw file ≠ Elements panel.

Also watch for: the browser can serve a **cached** old copy of a JS file. To see what's actually being served right now: DevTools → Network tab → enable "Disable cache" → hard refresh → check the file's **Response**

tab (not Sources, which can show a stale/pretty-printed copy).

---

9. Inline `style="..."` beats classes — and doesn't touch your CSS variables

Observation: A color-swatch button used `style="background: linear-gradient(135deg, rgb(...), rgb(...))"` directly, instead of Tailwind classes like `from-primary to-secondary`.

Concept: `from-primary`/`to-secondary`/`to-accent` are utility classes that just read CSS custom properties (`var(--color-primary)`, etc.) defined once in `:root`. Changing those three variables cascades to *every* element using those classes, site-wide — which is powerful, but only works if something is actually setting the variables.

An inline `style` attribute is a one-off override on a single element; it has higher specificity than a stylesheet rule but is completely disconnected from the `--color-*` variables — changing the variables won't affect an inline `background` set directly like that.

Fix — to make a color picker actually drive the whole page's theme:

```
document.documentElement.style.setProperty("--color-prima
document.documentElement.style.setProperty("--color-secon
document.documentElement.style.setProperty("--color-accen
```

This sets an inline style on `<html>`, which beats the `:root{}` block in the stylesheet (inline > external stylesheet, same specificity tier otherwise), and every `from-primary`/`to-secondary`/`to-accent` class on the page updates automatically — because they were only ever reading that variable.

---

10. Debugging habit: grep after a copy-paste

---

rename

Mistake: Copied the "alexandria" branch to build "tajawal" and "cairo" branches, renamed `e.target` → `target` in one but not the other two — left two stray `e.target` reads behind that silently broke those branches (clicking child elements returned `null` for `aria-checked`, matching neither `"true"` nor `"false"`, so nothing happened).

Concept: Copy-paste-and-rename is one of the most common sources of silent, hard-to-spot bugs, because the code still "looks right" at a glance.

Fix: After renaming a variable anywhere in a block you copy-pasted, `grep -n "oldName"` (or your editor's find-in-file) across that whole block before testing. Don't rely on eyeballing it.